PyTorch 핵심 치트시트

전체 흐름 코드의 줄 끝 번호 = 설명 카드 번호

기준 PyTorch 한국어 튜토리얼 「파이토치 기본 익히기」 · PyTorch 2.14로 실행 확인 · 2026-11-02

0 전체 흐름: 데이터 → 예측 → 손실 → 기울기 → 갱신

```
import torch
from torch import nn, optim
from torch.utils.data import Dataset, DataLoader
from torchvision import datasets, transforms
device = ("cuda" if torch.cuda.is_available()                    1
  else "mps" if torch.backends.mps.is_available()
  else "cpu")
lr, batch_size, epochs = 1e-3, 64, 5  # 사람이 정함              P
tf = transforms.ToTensor()                                       2
train_ds = datasets.FashionMNIST("data",
    train=True, download=True, transform=tf)
train_dl = DataLoader(train_ds, shuffle=True,                    2
                      batch_size=batch_size)
model = Net().to(device)          # 3번 카드                     3
loss_fn = nn.CrossEntropyLoss()                                  4
optimizer = optim.SGD(model.parameters(), lr=lr)                 5
for epoch in range(epochs):       # 에폭 반복
    model.train()                 # 학습 모드                    6
    for X, y in train_dl:         # 배치 반복                    2
        X, y = X.to(device), y.to(device)                        1
        pred = model(X)           # ① 예측                       3
        loss = loss_fn(pred, y)   # ② 손실                       4
        optimizer.zero_grad()     # ③ 초기화                     5
        loss.backward()           # ④ 기울기                     4
        optimizer.step()          # ⑤ 갱신                       5
    test(model)                   # 평가                         6
torch.save(model.state_dict(), "model.pth")                      7
```

한 배치에서 바뀌는 것 (batch_size=64)

train_ds[i]	샘플 1개: x `(1, 28, 28)`, y 정수
DataLoader	X `(64, 1, 28, 28)`, y `(64,)`
model(X)	로짓 `(64, 10)` (클래스 10개 점수)
loss_fn	배치 평균 손실, 스칼라 `()`
backward()	가중치마다 기울기 `p.grad` 계산
step()	가중치를 기울기 반대 방향으로 조금 이동

↻ 1 에폭 = 60000 / 64 → 938번 step

자주 하는 실수

1. `zero_grad()` 빠짐 → 기울기가 계속 쌓임
2. `CrossEntropyLoss`에 softmax 값을 넣음 → 로짓 그대로
3. 모델과 데이터의 장치가 다름 → 둘 다 `.to(device)`
4. 평가 때 `eval()`·`no_grad()` 빠짐

1 텐서 기본: shape 맞추기 `torch`

`x.shape` 가장 먼저 확인 · `x.to(device)` 모델과 같은 장치로

`x.reshape(64, -1)` -1은 자동 계산 · `x.flatten(1)` (N, 1, 28, 28) → (N, 784)

dim=1 →

dim=0 ↓		
1	2	3
4	5	6

`x.sum(dim=0)` → [5, 7, 9]
`x.sum(dim=1)` → [6, 15]

dim에 적은 축이 사라진다 → 예측 클래스는 **`pred.argmax(dim=1)`**

2 데이터: 미니배치로 나누기 `torch.utils.data`

```
class MyDS(Dataset):
    def __init__(self, X, y):
        self.X, self.y = X, y
    def __len__(self):            # 샘플 수
        return len(self.X)
    def __getitem__(self, i):     # 샘플 1개
        return self.X[i], self.y[i]
```

`DataLoader(ds, batch_size=64, shuffle=True)` 64개씩 묶고, 에폭마다 섞음(학습 데이터만)

`X, y = next(iter(train_dl))` 배치 하나 꺼내 shape 확인

3 모델: 예측 만들기 `torch.nn`

```
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Flatten(),
            nn.Linear(28*28, 512), nn.ReLU(),
            nn.Linear(512, 10))
    def forward(self, x):     # model(x)
        return self.layers(x)
```

`nn.Linear(in_features, out_features)` (N, in) → (N, out). 앞 층의 out = 다음 층의 in

`nn.ReLU()` 비선형. 없으면 Linear를 여러 층 쌓아도 Linear 하나와 같음

4 손실과 기울기 `torch.nn`

`nn.CrossEntropyLoss()` 분류: 로짓 (N,C), 정답 (N,) long

`nn.MSELoss()` 회귀: 예측과 정답의 제곱 오차

`loss.backward()` 모든 가중치의 기울기를 `p.grad`에 **더함** → 매번 `zero_grad()`

5 최적화: 가중치 갱신 `torch.optim`

`optim.SGD(model.parameters(), lr=lr)` $p \leftarrow p - lr \times p.grad$

`optim.Adam(model.parameters(), lr=1e-3)` 가중치마다 보폭을 조절. 처음 시도하기 무난

6 평가와 과적합 `torch`

```
def test(model):
    model.eval(); correct = 0  # Dropout 끔
    with torch.no_grad():      # 기울기 끔
        for X, y in test_dl:   # 안 본 데이터
            X, y = X.to(device), y.to(device)
            hit = model(X).argmax(1) == y
            correct += hit.sum().item()
    return correct / len(test_dl.dataset)
```

학습 손실은 내려가는데 평가 정확도가 떨어지면 과적합 → epochs 줄이기, **`nn.Dropout(p=0.5)`** 추가

7 저장·불러오기 `torch`

```
torch.save(model.state_dict(), "model.pth")
model.load_state_dict(torch.load("model.pth"))
```

P 파라미터 두 종류

가중치 `model.parameters()` : 학습이 바꾸는 값
하이퍼파라미터: 학습 전에 사람이 정하는 값

이름	키우면	줄이면
lr	빠름, 발산 위험	안정, 느림
batch_size	step↓, 메모리↑	step↑, 들쭉날쭉
epochs	과적합 위험↑	덜 학습
은닉 크기(512)	표현력↑, 과적합↑	단순한 패턴만